

Git and GitHub with VS Code

Step-by-Step Handout

Modern Computing and Python Programming (26ESCC106)

Manjunath Prasad H. R., Assistant Professor, Department of AIML, MITE

What You Will Learn

1. Install **Visual Studio Code (VS Code)** and **Git** on your computer.
2. Identify the parts of the VS Code window: menu bar, activity bar, side bar, editor, and the integrated PowerShell terminal.
3. Use the Git commands `git init`, `git config`, `git add`, `git commit`, `git remote`, and `git push`.
4. Create a **GitHub** account, sign in, and create a new repository.
5. Upload (push) your work to the repository on GitHub.
6. Explore the pushed repository on `github.com`: its home page, branches, commits, and the **Code** button.

Contents

1	Introduction: VS Code, Git, and GitHub	3
2	Installing VS Code (Windows)	3
3	Installing Git (Windows)	4
3.1	Verify the Installation	4
4	A Tour of the VS Code Window	5
4.1	Open a Project Folder	5
5	Creating a GitHub Account and Signing In	6
5.1	Sign Up (Create an Account)	6
5.2	Sign In (Log In)	6
6	How Git Tracks Your Work	6
7	Git Commands, Step by Step	7
7.1	Step 1: <code>git init</code> — Create a Repository	7
7.2	Step 2: <code>git config</code> — Tell Git Who You Are	7
7.3	Step 3: <code>git add .</code> — Stage the Changes	7
7.4	Step 4: <code>git commit -m "message"</code> — Save a Snapshot	8
7.5	Step 5: Create an Empty Repository on GitHub	8
7.6	Step 6: <code>git remote</code> — Connect to GitHub	8
7.7	Step 7: <code>git push origin main</code> — Upload to GitHub	9
8	The Complete Workflow	9
8.1	First Time (New Project)	9



8.2	Every Time After That	9
8.3	The Same Steps Using VS Code Buttons	10
9	Exploring Your Repository on GitHub	10
9.1	The Repository Home Page	10
9.2	Branches	11
9.3	Commits (History)	11
9.4	The Code Button	12
9.5	Demonstration: Watching a Push Appear on GitHub	13
10	Common Errors and How to Fix Them	13
11	Quick Reference	14
12	Practice Exercise	14

1. Introduction: VS Code, Git, and GitHub

Visual Studio Code (VS Code) is a free source-code editor from Microsoft. We use it to write Python programs and to run Git commands in its built-in terminal.

Git is a **version control system**. It runs on your own computer and records snapshots (versions) of your files, so you can see what changed, when, and by whom, and go back to an earlier version if needed.

GitHub is a website (github.com) that stores Git repositories **online**. It acts as a backup of your work and lets you share it with others.

Git versus GitHub: Git is the **tool** installed on your laptop; GitHub is the **online place** where you upload what Git has recorded.

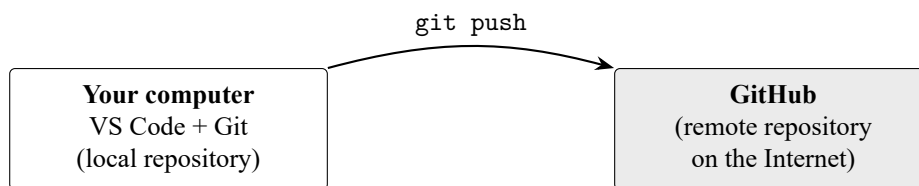


Figure 1: Git works on your computer; GitHub stores a copy online

Key terms:

- **Repository (repo):** A project folder whose history is tracked by Git.
- **Commit:** A saved snapshot of your files, with a message describing the change.
- **Branch:** A line of development. Our main branch is called `main`.
- **Remote:** A copy of the repository stored on another computer, such as GitHub.

2. Installing VS Code (Windows)

1. Open a web browser and go to <https://code.visualstudio.com>.
2. Click **Download for Windows**. The installer file (e.g., `VSCodeUserSetup-x64-... .exe`) is downloaded.
3. Double-click the downloaded file to start the installer.
4. **License Agreement:** Select **I accept the agreement** and click **Next**.
5. **Select Destination Location:** Keep the default folder and click **Next**.
6. **Select Start Menu Folder:** Keep the default and click **Next**.
7. **Select Additional Tasks:** Tick the following options and click **Next**:
 - Create a desktop icon.
 - Add “Open with Code” action to Windows Explorer file context menu.
 - Add “Open with Code” action to Windows Explorer directory context menu.
 - Register Code as an editor for supported file types.
 - **Add to PATH** (keep this ticked; it lets you type code in a terminal).
8. Click **Install** and wait for the installation to finish.
9. Tick **Launch Visual Studio Code** and click **Finish**.

Note

The installer screens may look slightly different in newer versions. If an option is not mentioned above, keep its default value.

3. Installing Git (Windows)

1. Go to <https://git-scm.com> and click **Download for Windows**.
2. Download the **64-bit Git for Windows Setup** file and double-click it.
3. **Information (license):** Click **Next**.
4. **Select Destination Location:** Keep the default and click **Next**.
5. **Select Components:** Keep the defaults and click **Next**.
6. **Choosing the default editor used by Git:** Select **Use Visual Studio Code as Git's default editor** and click **Next**.
7. **Adjusting the name of the initial branch in new repositories:** Select **Override the default branch name for new repositories** and make sure the name is **main**. Click **Next**.
8. **Adjusting your PATH environment:** Keep **Git from the command line and also from 3rd-party software** (recommended) and click **Next**.
9. For all the remaining screens, keep the default options and click **Next**.
10. Click **Install**, then **Finish**.

Why set the branch name to main?

We push our work using `git push origin main`. If Git names the first branch `master` instead, this command fails. Setting the default branch to `main` during installation avoids this problem. (If you missed this step, see Section 10.)

3.1 Verify the Installation

Close and reopen VS Code (so that it picks up the new PATH), open the terminal (**Terminal** → **New Terminal**), and type:

```
git --version
```

Expected output

```
git version 2.xx.x.windows.1
```

Any version number means Git is installed correctly. (The exact number depends on the version you downloaded.)

4. A Tour of the VS Code Window

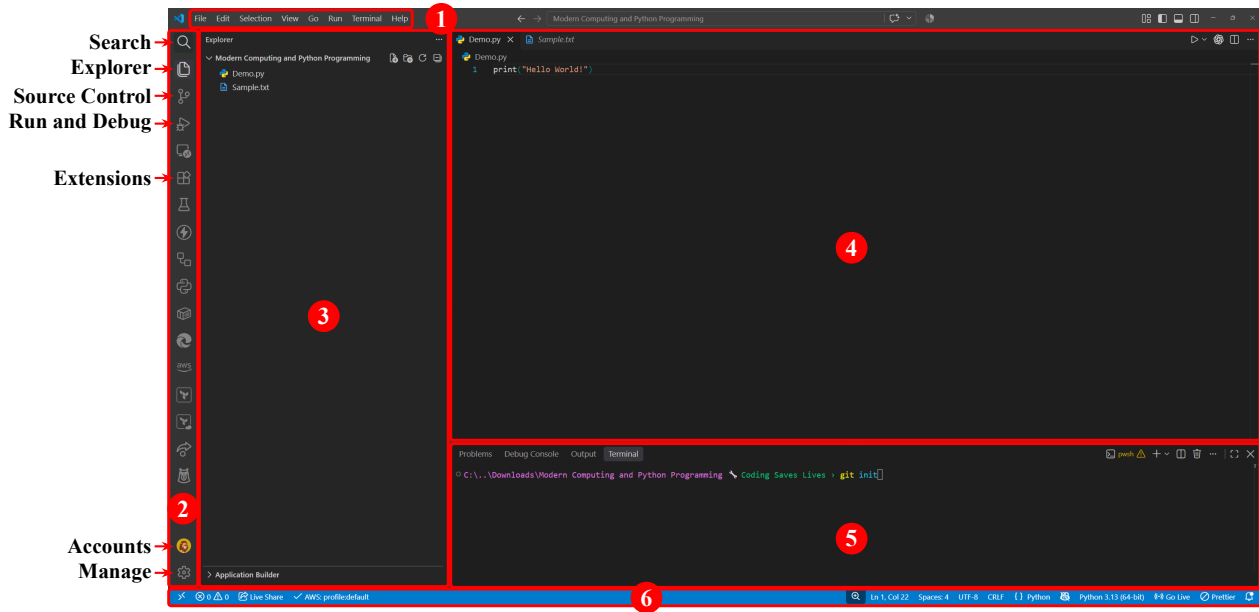


Figure 2: Main parts of the VS Code window — ① Menu bar, ② Activity bar, ③ Side bar (Explorer), ④ Editor, ⑤ Panel with the integrated terminal, ⑥ Status bar

1. **Menu bar:** The row of menus at the top: **File**, **Edit**, **Selection**, **View**, **Go**, **Run**, **Terminal**, **Help**. For example, **File** → **Open Folder** opens a project folder, and **Terminal** → **New Terminal** opens the terminal.
2. **Activity bar:** The narrow vertical bar of icons on the far left. Clicking an icon changes what the side bar shows. (Extra icons appear when you install extensions, e.g., Testing, Python, or AWS in Figure 2, and the order of icons can differ from computer to computer.)
 - **Explorer** (Ctrl+Shift+E): shows the files and folders of the opened project.
 - **Search** (Ctrl+Shift+F): searches for text across all files.
 - **Source Control** (Ctrl+Shift+G): the Git panel; shows **Changes** and **Staged Changes** and lets you commit.
 - **Run and Debug** (Ctrl+Shift+D): runs and debugs programs.
 - **Extensions** (Ctrl+Shift+X): installs add-ons such as the **Python** extension.
 - **Accounts and Manage** (bottom): sign in (e.g., to GitHub) and open settings.
3. **Side bar:** The panel next to the activity bar. Its content depends on the activity bar icon selected (Explorer, Source Control, etc.).
4. **Editor:** The large central area where you open and edit files. Each open file appears as a tab (e.g., `Demo.py` and `Sample.txt` in Figure 2).
5. **Panel with the integrated terminal:** The area at the bottom, with the tabs **Problems**, **Debug Console**, **Output**, and **Terminal**. The **Terminal** tab runs **PowerShell** (shown as `powershell` or `powershell1`; the default terminal on Windows) inside VS Code, so you can type Git commands without leaving the editor. Open it with **Terminal** → **New Terminal** or Ctrl+` (backtick).
6. **Status bar:** The thin bar at the very bottom. It shows errors and warnings, the cursor position (e.g., Ln 1, Col 22), the file's language (Python), and the Python version. Once the folder is a Git repository, it also shows the current branch (e.g., `main`).

4.1 Open a Project Folder

Git works on a **folder**. Before using Git, open your project folder in VS Code:

1. Create a folder, for example C:\Users\Student\MCPP.
2. In VS Code, select **File** → **Open Folder...**, choose the folder, and click **Select Folder**. If asked, click **Yes, I trust the authors**.
3. Create a file, e.g., hello.py, using the **New File** icon in the Explorer, and save it (Ctrl+S).
4. Open the terminal (Ctrl+`). The prompt shows that you are inside your folder:

Expected output

```
PS C:\Users\Student\MCPP>
```

5. Creating a GitHub Account and Signing In

5.1 Sign Up (Create an Account)

1. Open <https://github.com> and click **Sign up** (top-right).
2. Enter your **email address**. Use an email you can open now, because GitHub sends a verification code to it.
3. Create a **password**. Use a strong password and do not share it.
4. Choose a **username**. It becomes part of every repository address, e.g., github.com/<username>/<repository>, so choose a simple, professional name (e.g., your first and last name).
5. Select your **country/region** and email preferences, then click **Create account** (or **Continue**).
6. Complete the **verification puzzle** that checks that you are a human.
7. Open your email inbox, copy the **launch code** sent by GitHub, and enter it on the GitHub page.
8. Your account is now created. If GitHub asks optional questions about how you will use it, you may answer them or skip them. Choose the **Free** plan if a plan is offered.

5.2 Sign In (Log In)

1. Open <https://github.com> and click **Sign in**.
2. Enter your **username or email address** and your **password**, and click **Sign in**.
3. If GitHub asks for **device verification**, enter the code sent to your email.
4. If GitHub asks you to set up **two-factor authentication (2FA)**, follow the on-screen steps using an authenticator app on your phone, and **save the recovery codes** in a safe place.
5. When you are signed in, your profile picture (avatar) appears at the top-right of every GitHub page.

Note

GitHub changes its sign-up screens from time to time, so the exact wording may differ slightly. The information asked for (email, password, username, verification) remains the same.

6. How Git Tracks Your Work

Git moves your changes through **three local areas** and then to the **remote** repository on GitHub.

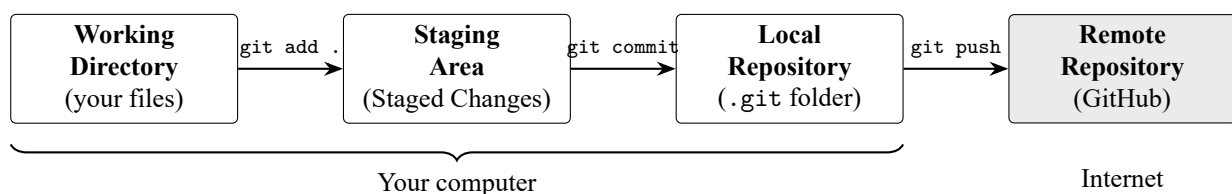


Figure 3: The path of a change — working directory → staging area → local repository → GitHub

1. **Working directory:** The files you see and edit in the Explorer. Edited files appear under **Changes** in the Source Control panel.
2. **Staging area:** A waiting area for the changes you want to include in the next commit. Staged files appear under **Staged Changes**.
3. **Local repository:** The hidden `.git` folder in your project, where Git stores every commit.
4. **Remote repository:** The copy of your repository on GitHub.

7. Git Commands, Step by Step

Type each command in the VS Code terminal **inside your project folder** and press Enter.

7.1 Step 1: `git init` — Create a Repository

Purpose: Turns the current folder into a Git repository. Git creates a hidden `.git` folder in which it will store the history of the project. Run it **once per project**.

```
git init
```

Expected output

```
Initialized empty Git repository in C:/Users/Student/MCPP/.git/
```

After this, the Source Control icon in the activity bar shows a number: the count of files that Git has noticed.

7.2 Step 2: `git config` — Tell Git Who You Are

Purpose: Every commit records the **name** and **email** of its author. Set them **once per computer**; the `--global` option applies them to all repositories on that computer.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

These commands print nothing when they succeed. To check the values you have set:

```
git config --global --list
```

Expected output

```
user.name=Your Name
user.email=your.email@example.com
```

Note

Use the **same email address** as your GitHub account, so that GitHub links the commits to your profile. Put the name in double quotes because it contains a space.

7.3 Step 3: `git add .` — Stage the Changes

Purpose: Moves all new and modified files from **Changes** to **Staged Changes**, i.e., prepares them for the next commit. The dot (`.`) means “everything in the current folder”.

```
git add .
```

This command normally prints nothing. On Windows you may see a harmless warning such as LF will be replaced by CRLF; it can be ignored.

In the Source Control panel, the files now move from **Changes** to **Staged Changes**.

7.4 Step 4: git commit -m "message" — Save a Snapshot

Purpose: Permanently records the staged changes in the local repository as a **commit**. The `-m` option supplies a short message describing what you changed.

```
git commit -m "Add hello.py program"
```

Expected output

```
[main (root-commit) 3f2a9c1] Add hello.py program
1 file changed, 1 insertion(+)
create mode 100644 hello.py
```

- `main` is the branch name; `root-commit` means this is the first commit.
- `3f2a9c1` is the short commit ID (yours will be different).
- After committing, the **Staged Changes** list in the Source Control panel becomes empty.

Writing good commit messages

Write a short message that says **what** changed, e.g., "Add factorial program" or "Fix input bug in marks.py". Avoid messages such as "update" or "done".

7.5 Step 5: Create an Empty Repository on GitHub

Before connecting your project to GitHub, a repository must exist there:

1. Sign in at <https://github.com> (Section 5).
2. Click **+** (top-right) → **New repository**.
3. Enter a **Repository name** (e.g., Modern-Computing-and-Python-Programming). Use hyphens instead of spaces.
4. Optionally, type a short **Description** of the project.
5. Choose **Public** (anyone can see it) or **Private** (only you and the people you invite can see it).
6. **Do not** add a README file, a `.gitignore`, or a license — the repository must be **empty**, because your files will come from your computer.
7. Click **Create repository**.
8. GitHub opens a **Quick setup** page. Make sure **HTTPS** is selected and copy the URL shown, which ends in `.git`. You will use it in Step 6.

7.6 Step 6: git remote — Connect to GitHub

Purpose: `git remote add` gives a short name to the GitHub repository's URL. By convention the name is `origin`.

```
git remote add origin https://github.com/ManjunathPrasad/Modern-Computing-and-Python-Programming.git
```

Replace the URL with the URL of your own repository. This command prints nothing when it succeeds.

To list the remotes that are connected:

```
git remote
```

Expected output

```
origin
```

To see the remote names together with their URLs, add `-v` (verbose):

```
git remote -v
```

Expected output

```
origin https://github.com/ManjunathPrasad/Modern-Computing-and-Python-Programming.git (fetch)
origin https://github.com/ManjunathPrasad/Modern-Computing-and-Python-Programming.git (push)
```

7.7 Step 7: git push origin main — Upload to GitHub

Purpose: Sends the commits of the local main branch to the remote named origin (GitHub).

```
git push origin main
```

First push only: A window titled **Connect to GitHub** appears. Click **Sign in with your browser**, log in to GitHub, and click **Authorize**. Git remembers this login, so you will not be asked again on the same computer.

Expected output

```
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 245 bytes | 245.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ManjunathPrasad/Modern-Computing-and-Python-Programming.git
 * [new branch]      main -> main
```

Refresh the repository page on GitHub: your files and your commit message now appear there. Section 9 explains what you see on that page.

8. The Complete Workflow

8.1 First Time (New Project)

```
git init
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
git add .
git commit -m "First commit"
git remote add origin https://github.com/<username>/<repository>.git
git push origin main
```

8.2 Every Time After That

Once the project is set up, the daily cycle has only three commands:

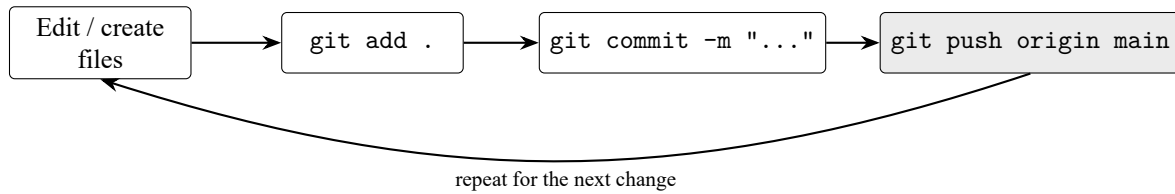


Figure 4: The everyday Git cycle

```

git add .
git commit -m "Describe your change"
git push origin main
  
```

8.3 The Same Steps Using VS Code Buttons

The Source Control panel (Ctrl+Shift+G) can do the same work without typing commands:

Command	Equivalent action in the Source Control panel
git init	Click Initialize Repository (shown when the folder is not yet a repository).
git add .	Click the + (Stage All Changes) icon next to the Changes heading.
git commit -m "..."	Type the message in the box at the top and click Commit .
git push origin main	Click Sync Changes / Publish Branch , or ... → Push .

9. Exploring Your Repository on GitHub

After a few pushes, open your repository on github.com. The screenshots below are of the course repository after **four commits** were pushed.

9.1 The Repository Home Page

Address: <https://github.com/<username>/<repository>>

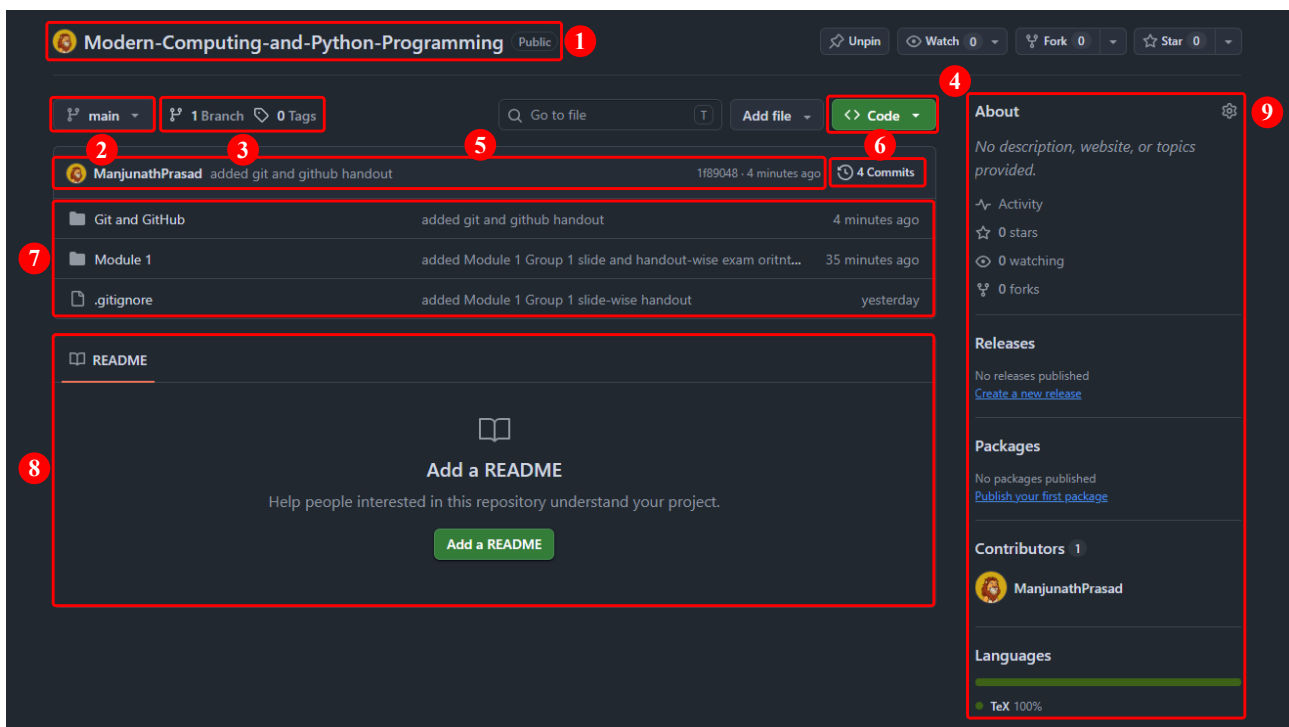


Figure 5: Repository home page after four pushes

1. **Repository name and visibility:** The name of the repository and a **Public** (or **Private**) label. In the page header, the owner's username (ManjunathPrasad) appears before the repository name.
2. **Branch selector:** Shows the branch being viewed (main). Click it to switch to another branch.
3. **Branches and tags:** **1 Branch** shows how many branches the repository has; click it to open the branches page (Figure 6). **0 Tags** shows that no version has been tagged yet.
4. **Code button:** Gives the repository URL and a ZIP download (Figure 8).
5. **Latest commit:** The author (ManjunathPrasad), the commit message (added git and github handout), the short commit ID (1f89048), and how long ago it was pushed.
6. **Commit count:** **4 Commits** is the total number of commits on this branch. Click it to open the commit history (Figure 7).
7. **Files and folders:** The contents of the repository. Each row shows the name, the message of the **last commit that changed it**, and when. Note that build files such as .aux and .log do not appear, because .gitignore told Git to ignore them.
8. **README area:** GitHub displays the README.md file here. This repository has none yet, so GitHub offers an **Add a README** button.
9. **About sidebar:** The description, stars, watchers, forks, releases, packages, **contributors** (people who have committed), and **languages** (detected from the file types; here TeX 100%).

9.2 Branches

Address: <https://github.com/<username>/<repository>/branches> (or click **1 Branch** in Figure 5)

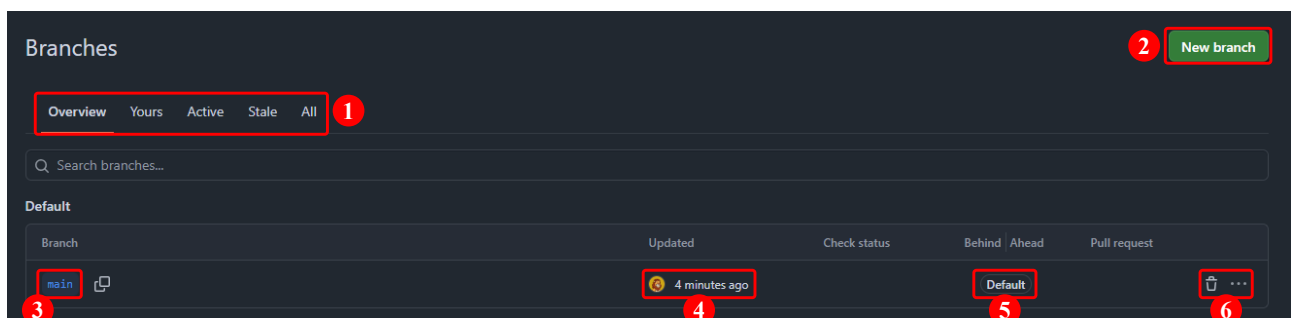


Figure 6: The branches page

1. **Tabs:** Overview, Yours, Active, Stale, and All filter the list of branches.
2. **New branch:** Creates a new branch on GitHub.
3. **Branch name:** The repository has one branch, main — the branch we pushed with `git push origin main`.
4. **Updated:** When the branch last received a commit, and by whom (avatar).
5. **Default:** main is the **default branch**: GitHub shows it on the home page and uses it unless another branch is chosen. The **Behind/Ahead** columns compare other branches with the default branch.
6. **Delete and more options:** The bin icon deletes a branch and ... shows more actions. (The default branch cannot be deleted.)

9.3 Commits (History)

Address: <https://github.com/<username>/<repository>/commits/main> (or click **4 Commits** in Figure 5)

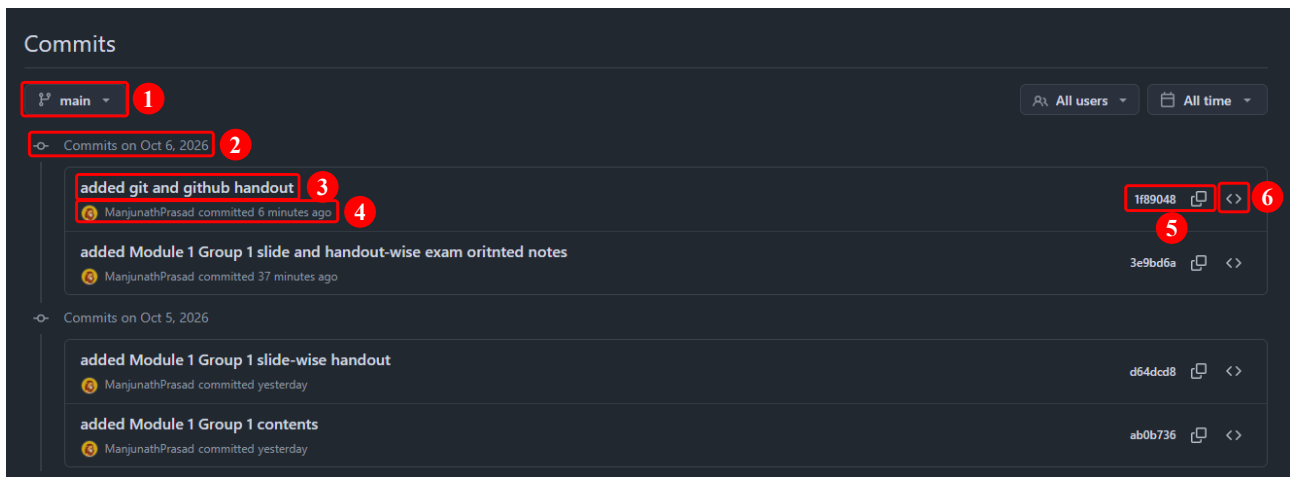


Figure 7: The commit history of the main branch

1. **Branch selector:** The history shown is for the main branch.
2. **Date groups:** Commits are grouped by the day they were made (**Commits on Oct 6, 2026**). The **newest commit is at the top**.
3. **Commit message:** Exactly the text given in `git commit -m "..."`. Click it to see the changes in that commit: added lines in **green** and removed lines in **red**.
4. **Author and time:** Who made the commit (from `git config user.name`) and when.
5. **Commit ID:** The short commit ID (1f89048) — the same ID that `git commit` printed in the terminal. The icon next to it copies the full ID.
6. **Browse files (<>):** Shows the whole repository as it was at that commit.

Why good commit messages matter

The commit history is a diary of the project. Clear messages such as added Module 1 Group 1 slide-wise handout tell everyone what changed in each step without opening the files.

9.4 The Code Button

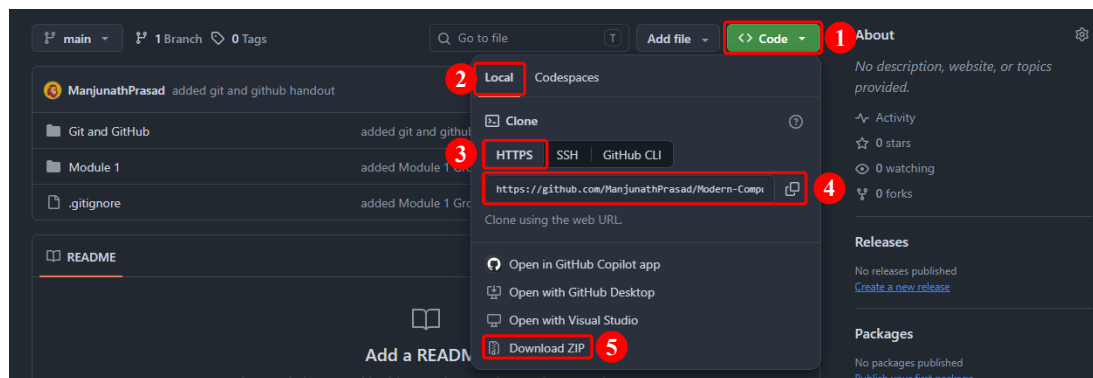


Figure 8: The Code button menu

1. **Code button:** The green button on the repository home page.
2. **Local tab:** Options for getting the code onto your own computer. (**Codespaces** opens the code in an online editor instead.)
3. **HTTPS tab:** Select HTTPS — the same type of URL used in this handout.
4. **Repository URL and copy button:** The URL ending in `.git`. This is the URL used in `git remote add origin <URL>`. Click the copy icon to copy it.
5. **Download ZIP:** Downloads all the files as a single ZIP file (without the Git history).

9.5 Demonstration: Watching a Push Appear on GitHub

1. Open the repository home page (Figure 5) and note the number of commits (e.g., **4 Commits**).
2. In VS Code, edit a file or add a new one, and save it.
3. In the terminal, run:

```
git add .
git commit -m "Add sum.py program"
git push origin main
```

4. Refresh the GitHub page. The commit count increases by one (**5 Commits**), and the latest-commit row shows the new message.
5. Click the commit count. The new commit is at the top of the history (Figure 7), with the same commit ID that the terminal printed.
6. Click the commit message to see exactly which lines were added or changed.
7. Open **1 Branch** (Figure 6): the **Updated** time of main has changed to “now”.

10. Common Errors and How to Fix Them

Error message	Cause and fix
git : The term 'git' is not recognized...	Git is not installed, or VS Code was open during installation. Close and reopen VS Code (or restart the computer) and try again.
fatal: not a git repository	You are not inside a repository. Check the folder in the prompt, open the correct folder, or run <code>git init</code> .
Author identity unknown ...Please tell me who you are.	The name and email are not set. Run the two <code>git config --global</code> commands (Step 2).
nothing to commit, working tree clean	No changes since the last commit. Edit and save a file first, then run <code>git add .</code> again.
error: src refspec main does not match any	Either no commit has been made yet, or your branch is called <code>master</code> . Make a commit first; if the status bar shows <code>master</code> , rename the branch with <code>git branch -M main</code> , then push again.
error: remote origin already exists.	<code>origin</code> was added earlier. To change its URL, run <code>git remote set-url origin <new-URL></code> .
! [rejected] main -> main (fetch first)	The GitHub repository already contains files (e.g., a README created on the website). Create a new empty repository (Step 5) and add that URL instead.
Authentication failed / Permission denied	You are signed in to a different GitHub account, or the URL is wrong. Check the URL with <code>git remote -v</code> and sign in with the account that owns the repository.

11. Quick Reference

Command	What it does	How often
<code>git --version</code>	Shows the installed Git version	Once
<code>git init</code>	Makes the current folder a Git repository	Once per project
<code>git config --global user.name "Name"</code>	Sets the author name for commits	Once per computer
<code>git config --global user.email "email"</code>	Sets the author email for commits	Once per computer
<code>git config --global --list</code>	Shows the configured settings	When checking
<code>git add .</code>	Stages all changes (Changes → Staged Changes)	Every change
<code>git commit -m "message"</code>	Saves the staged changes as a commit	Every change
<code>git remote / git remote -v</code>	Lists remotes (with URLs when -v is used)	When checking
<code>git remote add origin <URL></code>	Connects the repository to GitHub	Once per project
<code>git push origin main</code>	Uploads commits to GitHub	Every change

12. Practice Exercise

1. Create a folder named MCPP-Lab and open it in VS Code.
2. Create a file `hello.py` containing `print("Hello, MITE")` and save it.
3. Run `git init` and observe the `.git` message.
4. Set your name and email with `git config --global` and verify them with `git config --global --list`.
5. Run `git add .` and observe the file moving to **Staged Changes** in the Source Control panel.
6. Commit with the message "Add hello program".
7. Create an empty repository named MCPP-Lab on GitHub.
8. Connect it using `git remote add origin <your-URL>` and check it with `git remote -v`.
9. Push using `git push origin main` and confirm that `hello.py` appears on GitHub.
10. Add a second file `sum.py` that prints the sum of two numbers. Add, commit, and push it, and check that GitHub now shows **2 Commits**.
11. On GitHub, open **2 Commits** and identify the commit ID, message, author, and time of each commit. Compare the IDs with the ones printed by `git commit` in your terminal.
12. Open the branches page and confirm that `main` is the **Default** branch and shows the latest update time.